MF20006: Introduction to Computer Science

Lecture 1: Numbers and Computation

Hui Xu

xuh@fudan.edu.cn

Outline

1. Integers

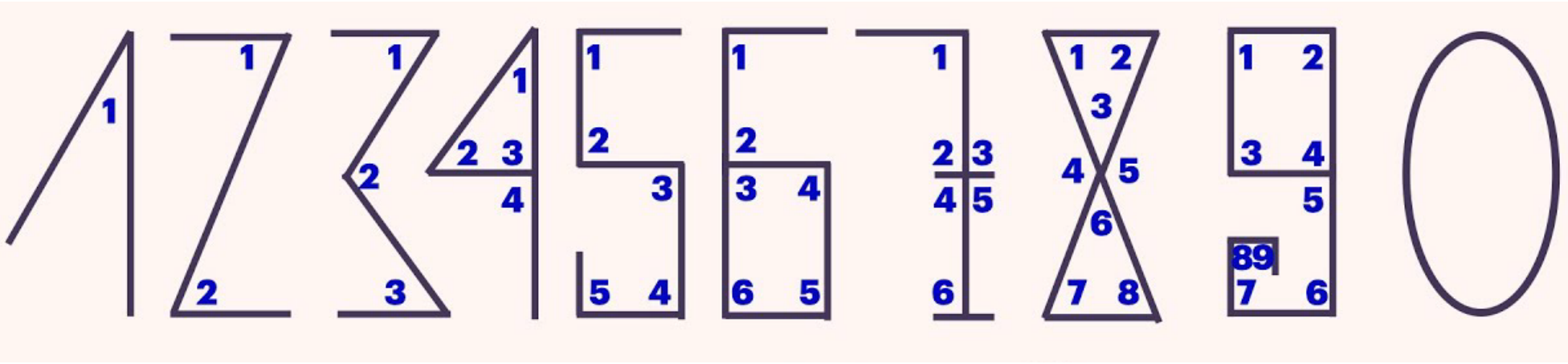
2. Computation

3. Floating-point Numbers

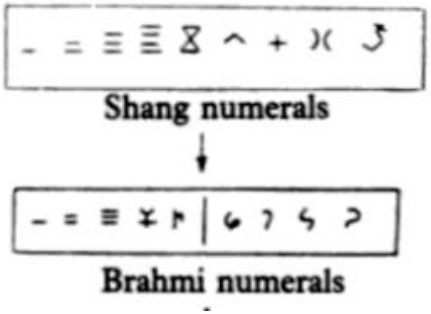
1. Integers

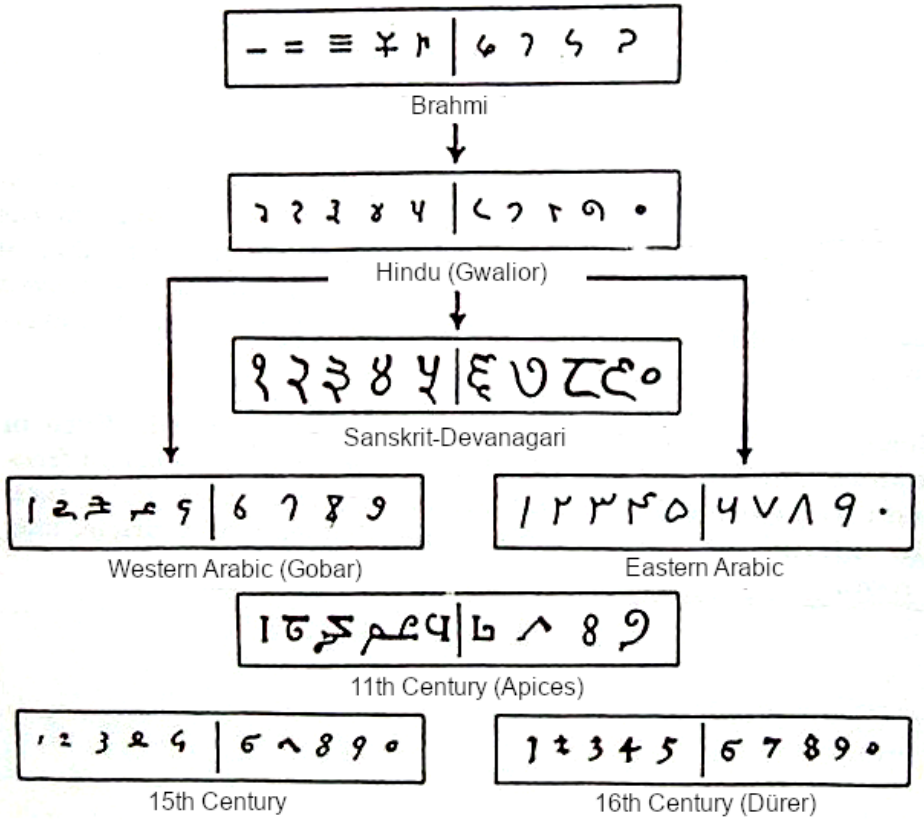
Arabic Numerals

- Myth: based on the numbers of angles.



Arabic Numerals





https://en.wikipedia.org/wiki/Arabic_numerals

Decimal Numbers

- **Each digit represents a power of 10.**

$$123 = 10^2 + 2 * 10^1 + 3 * 10^0$$

- **We can generalize this idea to any base *n*.**
  - Each position represents a power of *n*.

$$(123)_n = 1 * n^2 + 2 * n^1 + 3 * n^0$$

Binary Numbers (base-2)

- Each binary digit, or bit, represents a power of 2.
- In computer, we represent numbers with binary numbers.

$$(1011)_2 = 1 * 2^3 + 0 * 2^2 + 1 * 2^1 + 1 * 2^0 = 11$$

Binary Numbers in Computer Storage

❑Computer storage consists of consecutive bits.

...00000000000**00001000**0001000000011000001000000101...

Address: p

❑How to read and interpret these bits as numbers?

- ➢start: via the memory address.
- ➢end: assuming a fixed length of bits for each number, e.g., 8-bit, 32-bit.

Represent Natural Numbers with 8 Bits (1 Byte)

Decimal	**Fixed-length** (8 bit/1 byte)
0	00000000
1	00000001
2	00000010
3	00000011
4	00000100
5	00000101
6	00000110
7	00000111
8	00001000
9	00001001
10	00001010
11	00001011
12	00001100
…	…

Representing Binaries in Hexadecimal Format

Fixed-length (8 bit/1 byte)	**Hex** (base 16)
00000000	0
00000001	1
00000010	2
...	...
00000111	7
00001000	8
00001001	9
00001010	A
00001011	B
00001100	C
00001101	D
00001110	E
00001111	F
00010000	10
…	…

Exercise

- **What is the decimal value of the following binaries:**
  - 1111 1111
  - 1000 0000
- **What is the binary representation for:**
  - $(100)_{10}$
  - $(1024)_{10}$ (also known as 1 kilo)

More Quantifiers

- 1 mega (M): 2^{20}
- 1 giga (G): 2^{30}
- 1 tera (T): 2^{40}
- 1 peta (P): 2^{50}
- 1 exa (E): 2^{60}
- 1 zetta (Z): 2^{70}
- 1 yotta (Y): 2^{80}
- ...

Exercise

- **Given any decimal number X, how to calculate the binary representation step by step? Express your idea "formally",**
  - e.g., via pseudo code or natural language

Solve the problem via LLM

❑Can LLM reliably solve the problem?

- ➢Try to find an adversarial case.
- ➢Compare the LLM generated result with the ground truth.
  - e.g., https://www.rapidtables.com/convert/number/binary-to-decimal.html

❑Solve the problem via coding agent:

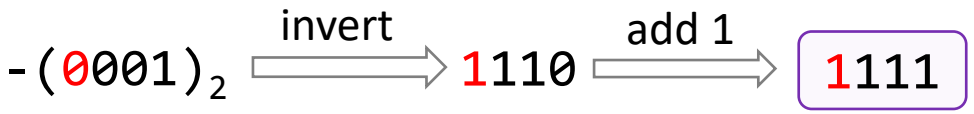
- ➢Install any popular coding agent, e.g., opencode, codex, claude code.

How to Represent Negative Numbers?

- **Option 1: Simply employ the first bit as the sign bit.**
  - $(0011)_2 = 3, (1011)_2 = -3$
  - Issue: 0 has two representations.
- **Option 2: Complement-based approach by flipping all bits.**
  - e.g., one's complement: $x + y = 2^n - 1$
  - $(1000)_2 = -(0111)_2 = -7$
  - $(1011)_2 = -(0100)_2 = -4$
  - $(1111)_2 = -(0000)_2 = -0$
  - Issue: seems no benefit; 0 still has two representations.
- **Two's complement:** $x + y = 2^n$
  - $(1000)_2 = -(0111 + 1)_2 = -8$
  - $(1111)_2 = -(0000 + 1)_2 = -1$

Representing Negative Integers with Two's Complement

- **Employ the leftmost bit as the sign bit.**
  - 0 for positive numbers.
  - 1 for negative numbers.
  - Padding 0 or 1 to a fixed size, *e.g.,* 4 bits
- **Obtain the two's complement of numbers.**
  - Invert all the bits of the positive number, then add one.
  - Or sub one first, then invert all bits.



Decimal	Binary
-8	1000
-7	1001
-6	1010
-5	1011
-4	1100
-3	1101
-2	1110
-1	1111
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111

Benefits of Two's Complement

- Turn all subtractions in to adds => Reuse the adder (circuit).

$7 - 5 = 7 + (-5)$

$$\begin{array}{rr} & 0111 \\ + & 1011 \\ \hline & \cancel{1}0010 \end{array}$$

$2 - 5 = 2 + (-5)$

$$\begin{array}{rr} & 0010 \\ + & 1011 \\ \hline & 1101 \end{array}$$

Why?

To Prove: $\big(x + (-y)\big) \bmod 2^n = (x - y) \bmod 2^n$

According to the definition of two's complement:

$$y + (-y) = 2^n$$

Therefore,

$$\big(x + (-y)\big) \bmod 2^n$$

$$= \big(x + (2^n - y)\big) \bmod 2^n$$

$$= (x - y) \bmod 2^n$$

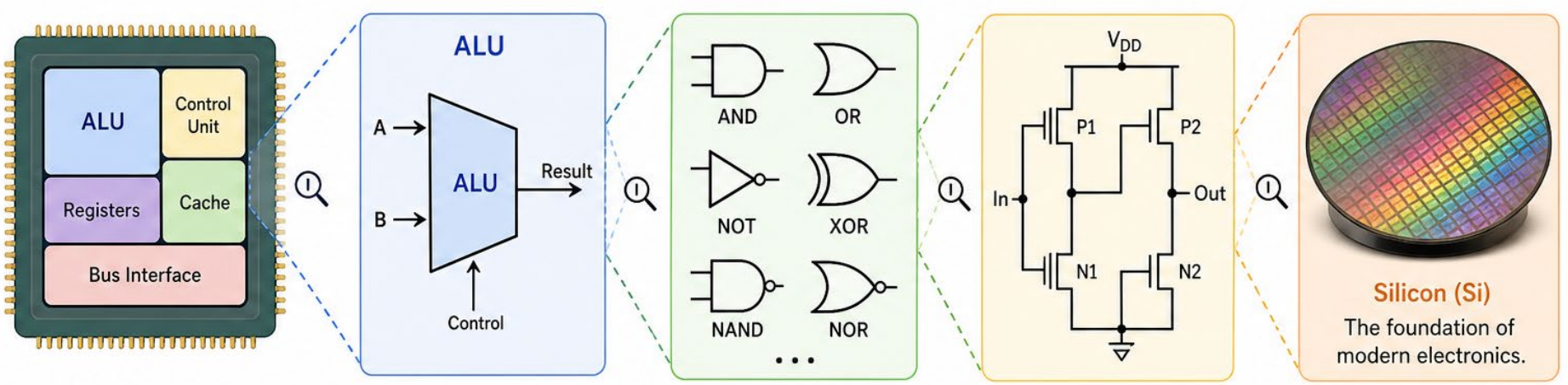
Exercise

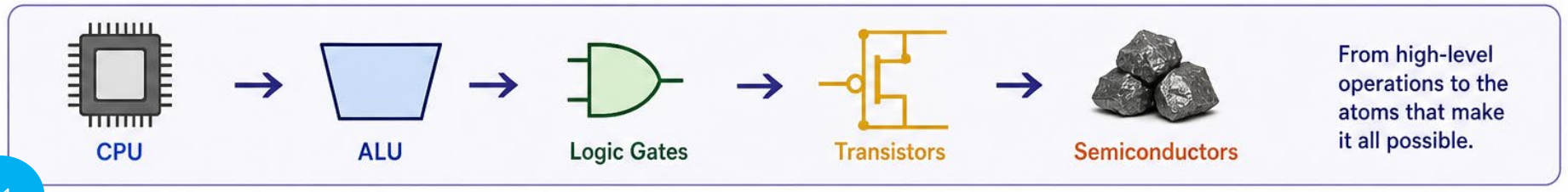
- What is the largest signed integer that 16 bits can represent?
- What is the smallest signed integer that 16 bits can represent?

2. Computation

Computation Unit

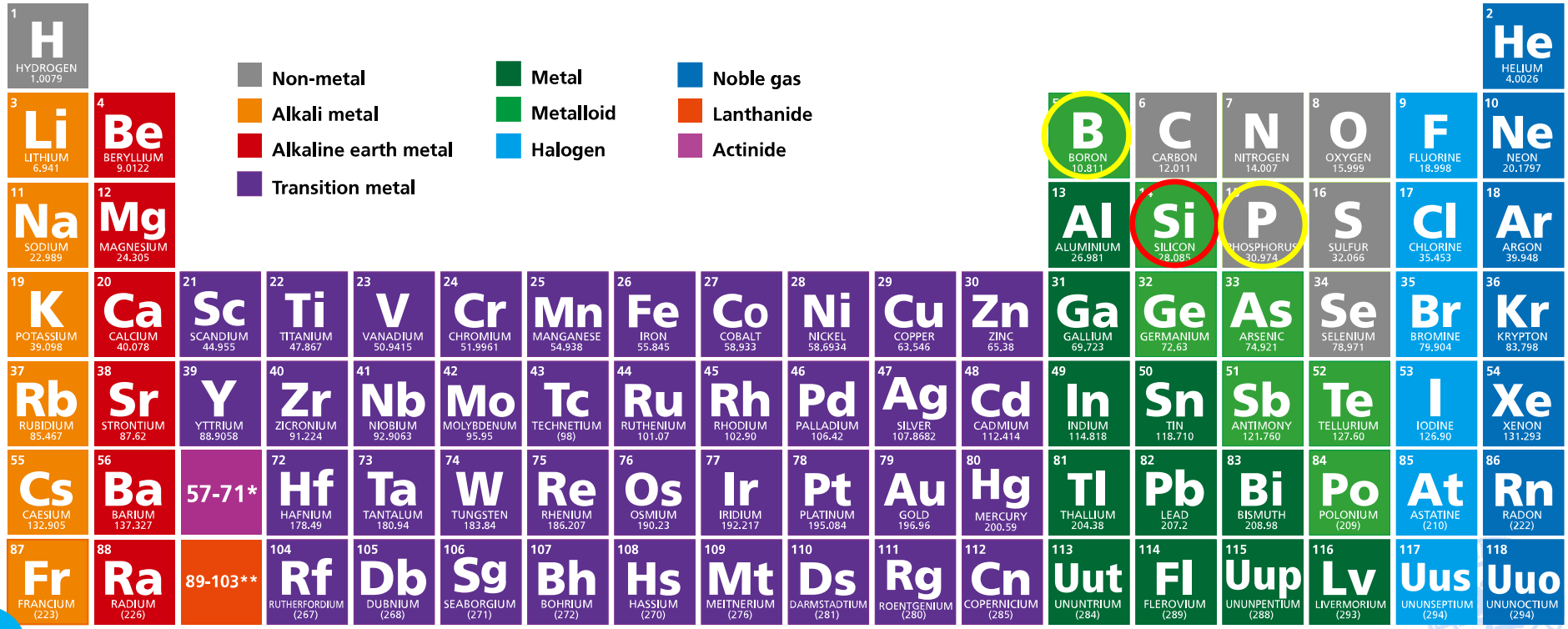
- CPUs are made with ALU (Arithmetic Logic Unit) and others.
- The ALU is built using logic gates.
- Logic gates are implemented with transistors.
- Transistors are made from semiconductors, primarily silicon.





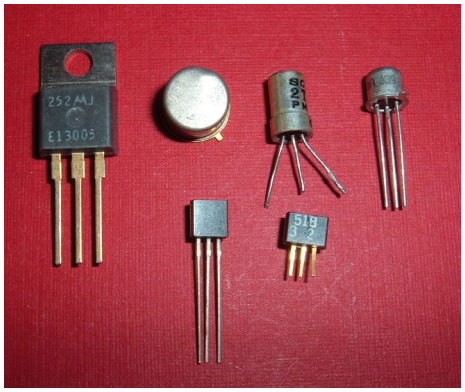
Semiconductor, Why Silicon?

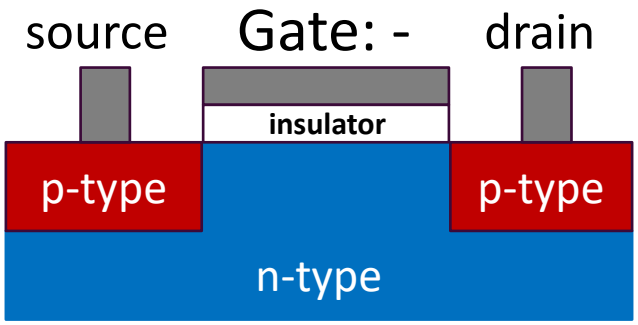
- ❑Pure silicon has limited conductivity.
- ❑Doping: Adding small amounts of impurities to change conductivity.
  - ➢P-type: Add atoms with 3 valence electrons *e.g.,* boron.
  - ➢N-type: Add atoms with 5 valence electrons (*e.g.,* phosphorus)



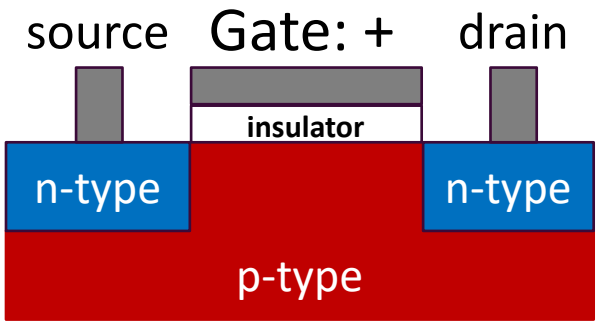
Transistors

- A transistor is an electronic switch that controls the flow of current.
- Types of Transistors:
  - PMOS (P-type MOSFET): Conducts when gate is low.
  - NMOS (N-type MOSFET): Conducts when gate is high.





PMOS



NMOS

Logical Gate: NOT

❑When no voltage is applied to the gate (A = 0)

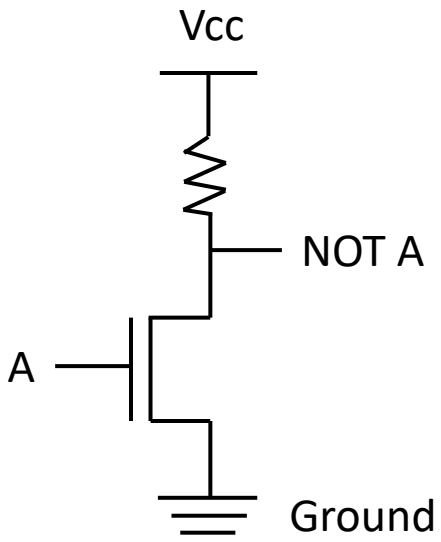
- ➢The NMOS is not conductive (OFF).
- ➢The output (Not A) is pulled up to VCC: Not A = 1

❑When a positive voltage is applied to the gate (A = 1)

- ➢The NMOS becomes conductive (ON).
- ➢The output (Not A) is pulled down to Ground: Not A = 0

Truth Table:

A	Not A
0	1
1	0



NMOS NOT

Vcc: Voltage Common Collector (+ terminal)

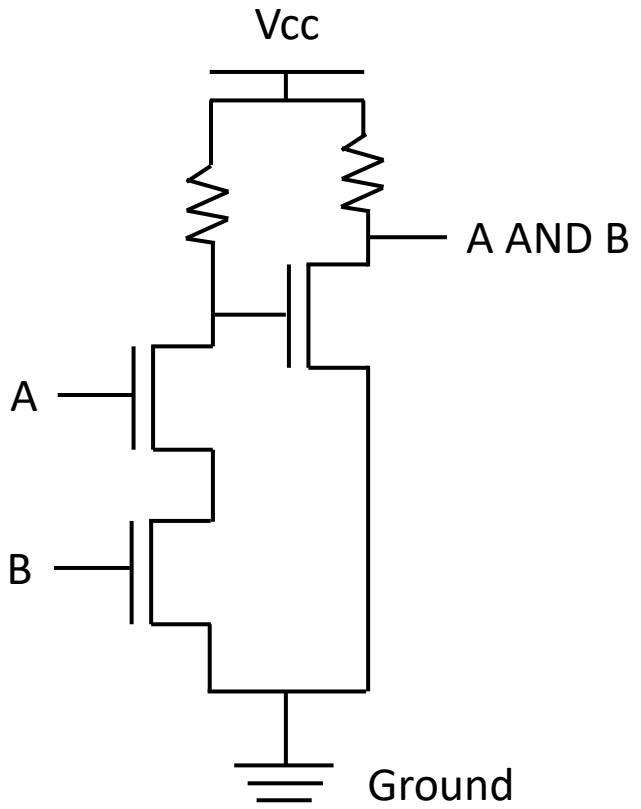
Logical Gate: AND

❑When no voltage is applied to the gates (A = 0, B = 0)

- ➢The third gates has voltage and becomes conductive.
- ➢The output (A AND B) is pulled down to VCC: Not A = 1

Truth Table:

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

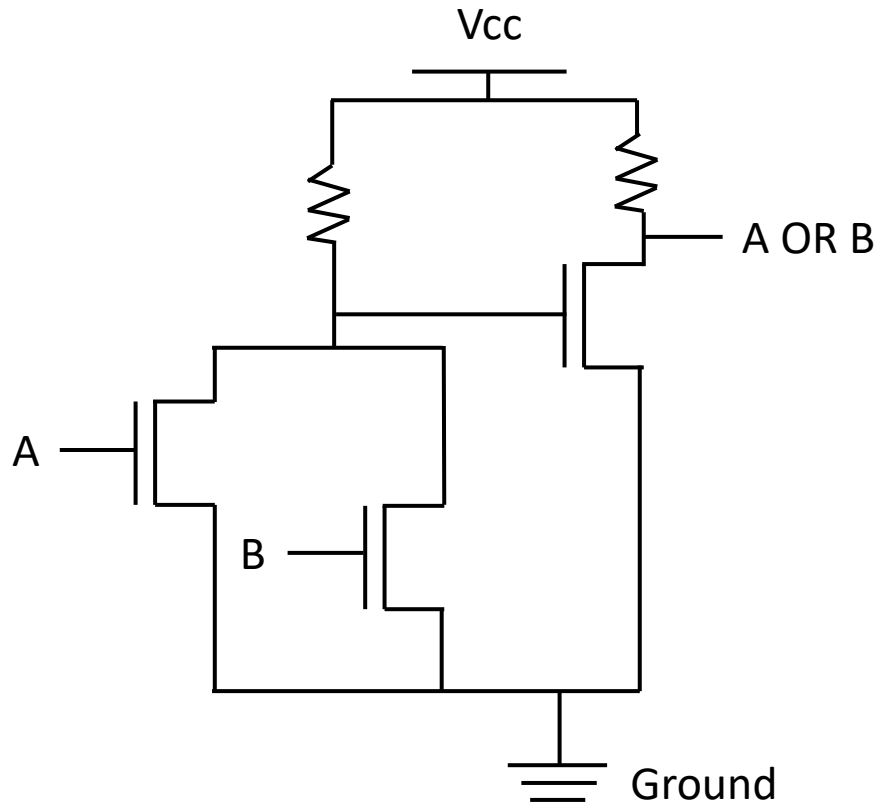


NMOS AND

Logical Gate: OR

Truth Table:

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

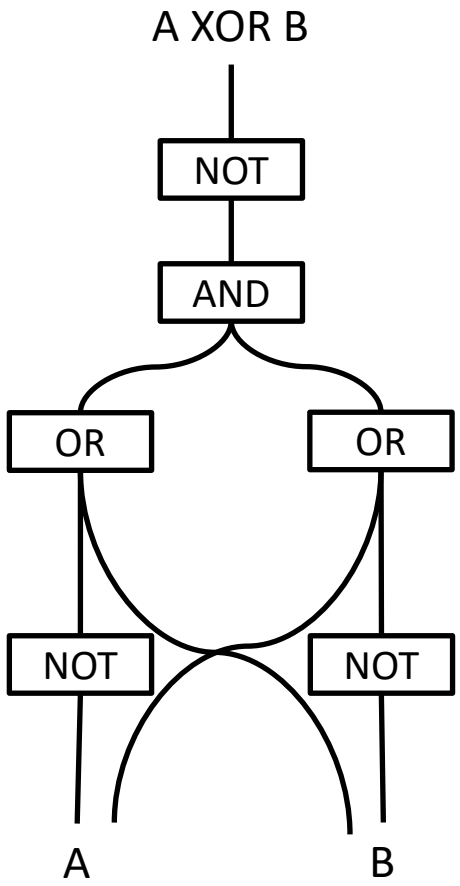


NMOS OR

Logical Gate: XOR

Truth Table:

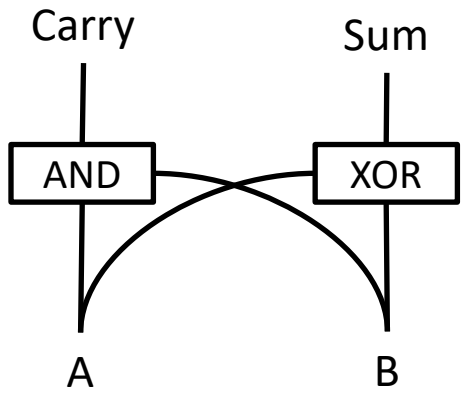
A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0



Compose a Half Adder with Logical Gates

- Half adder has no carry input.

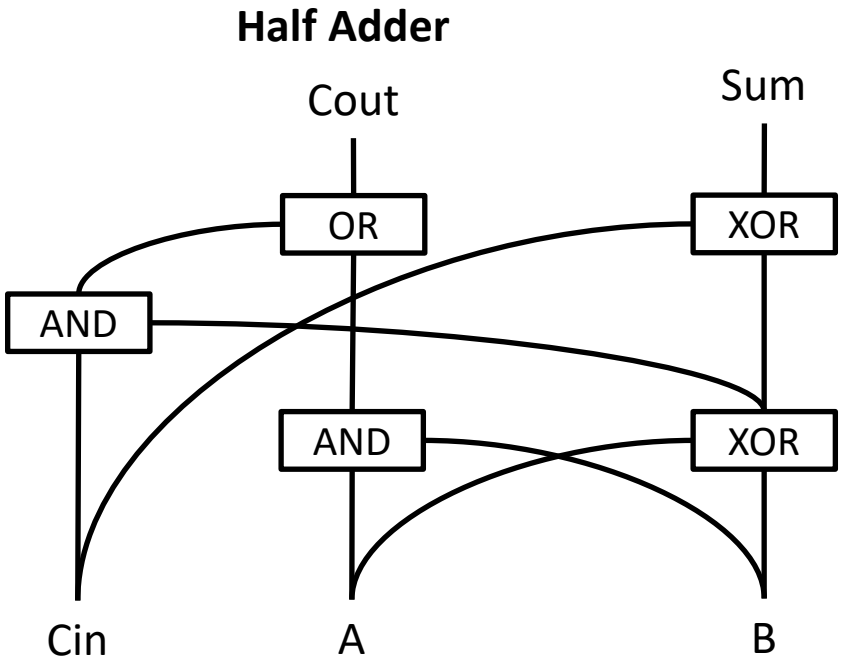
Inputs		Outputs	
A	B	C_{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



Half Adder

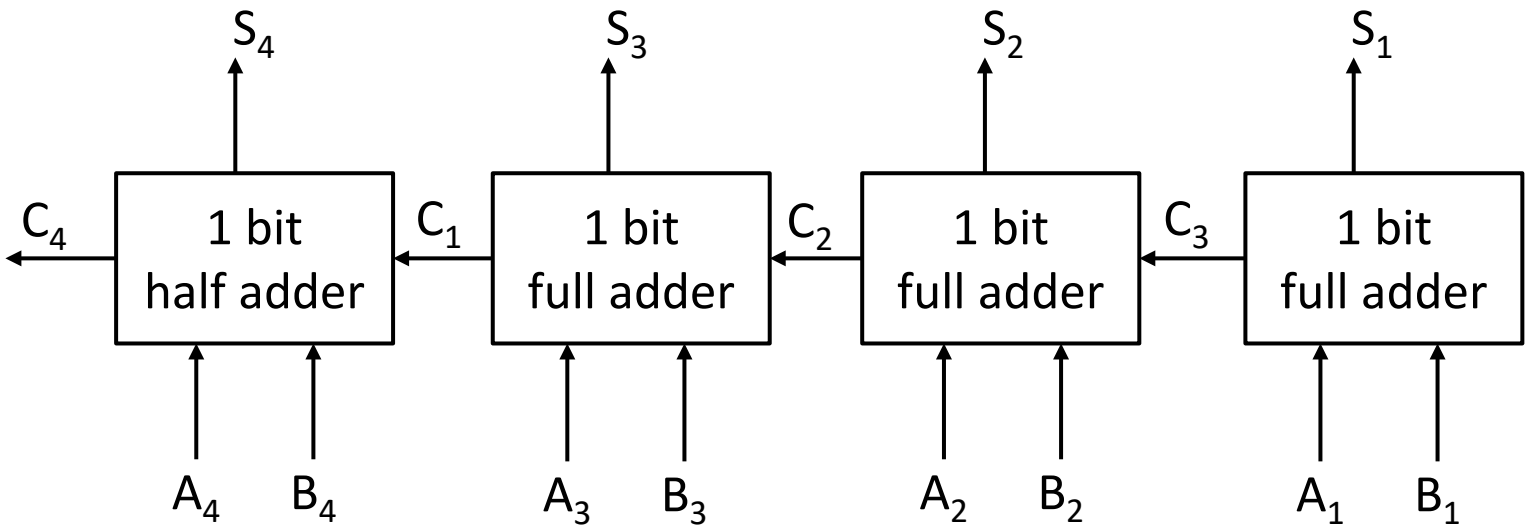
Full Adder

Inputs			Outputs	
A	B	C_{in}	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



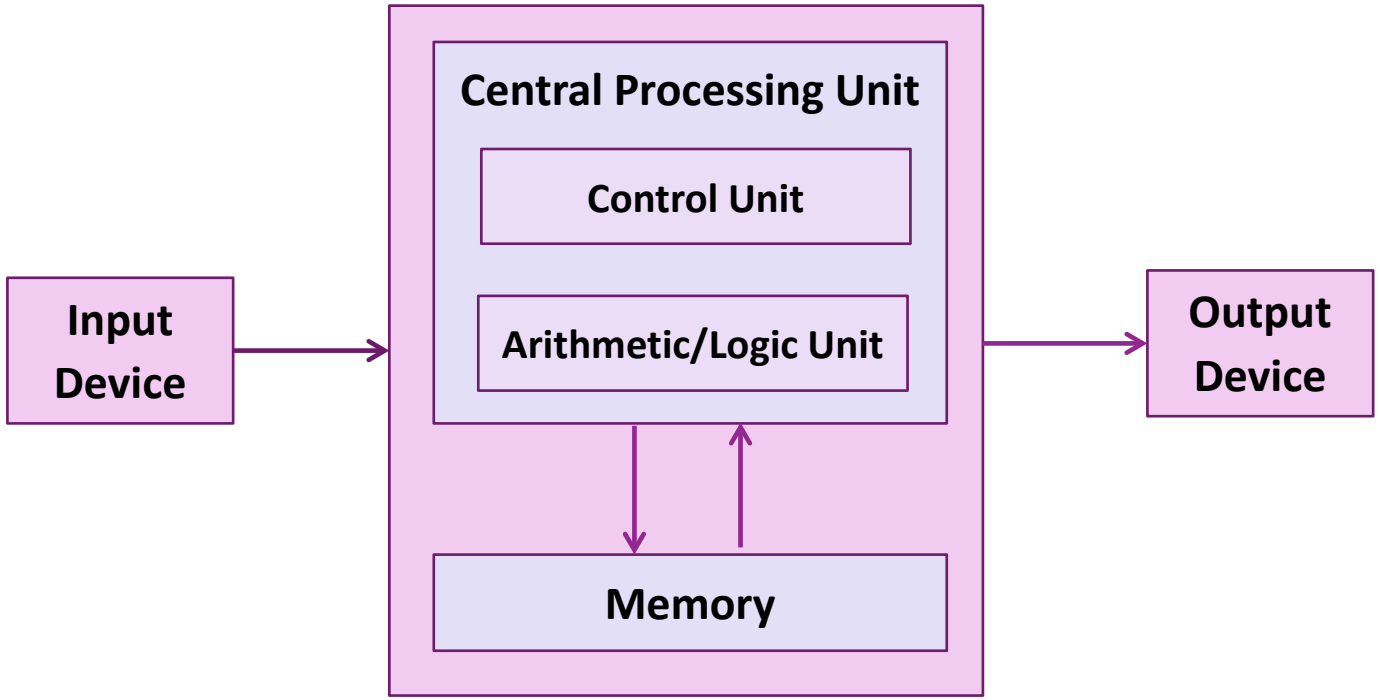
Full Adder (with carrier input)

Adder of 4 Bits



CPU and Von Neumann Model

- Control unit: fetch instructions from memory at the address specified by the program counter.
- ALU: perform the operation specified by the instruction, write the results to memory or registers.



3. Floating-point Numbers

More Issues to be Considered

- How to represent fractional numbers?
- What if the number is very large?

Representing Decimals: Fixed-point Method

- Allocate a fixed number of bits for the integer part and the fractional part.

$$(1000.0100)_2 = 2^3 + 2^{-2} = 8.25$$

- Limited range and fixed precision

Representing Decimals: Floating-point Method

- Shift the decimal point based on the number.
- If the number is large, use more bits for the integer part.

$$1000000.1$$

- If the number is small, use more bits for the fractional part

$$1.0000001$$

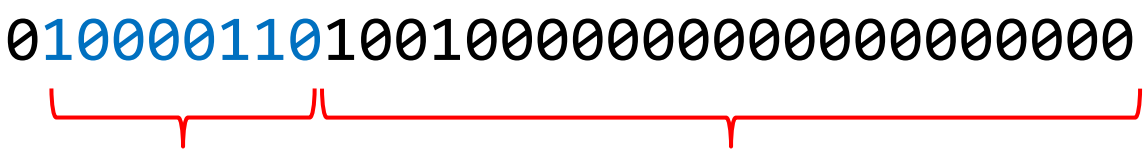
- How to achieve?
  - Some conventions/standard need to be established for encoding/decoding.

Floating-point Numbers Defined in IEEE 754

❑Meaning of bits for 32-bit single-precision floating-point numbers:

- ➢32 (leftmost): sign bit.
- ➢24-31: exponent bits, to represent a wide range of values.
- ➢1-23: mantissa bits.

❑Evaluation: $2^{exp-bias} * mantissa$



exponent (8 bits)

mantissa (23 bits)

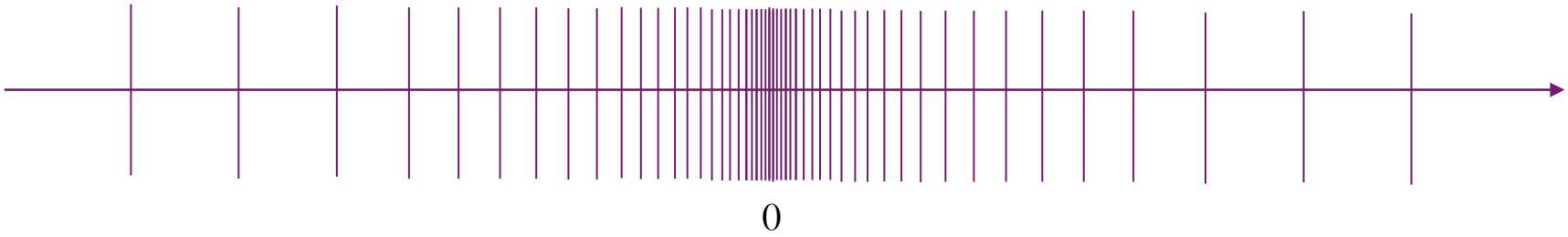
$2^7 + 2^2 + 2^1 - 127 = 7$

$1 + 2^{-1} + 2^{-4} = 1.5625$

$$2^7 * 1.5625 = 200$$

Why do we subtract a bias 127?

Non-uniform Distribution of Floating-point Numbers



Exercise

- What is the value of the following floating-point numbers
  - 01111111000000000000000000000000
  - 01000000011000000000000000000000
  - 11000000101100000000000000000000

Conversion to Floating-point Numbers

- What is the floating-point representation for 11.25?

Integer Part	Fractional Part
11/2 = 5 + 1	0.25 * 2 = 0.5 + 0
5/2 = 2 + 1	0.50 * 2 = 0.0 + 1
2/2 = 1 + 0	
1/2 = 0 + 1	

1011.01

1+.01101 with exp = 3

0 10000010 01101000000000000000000

3+127

Exercise

- **Which of the following numbers can be represented by floating-point numbers without precision loss?**
  - 0.1
  - 0.2
  - 0.3
  - 0.4
  - 0.5

Solve the problem via LLM

❑Can LLM reliably solve the problem?

- ➢Try to find an adversarial case.
- ➢Compare the LLM generated result with the ground truth.
  - https://www.h-schmidt.net/FloatConverter/IEEE754.html

❑Solve the problem via coding agent:

- ➢Install any popular coding agent, e.g., opencode, codex, claude code.

Beside Numbers, All Data are Stored in Computers as Bits

- Text/Documents
- Images
- Videos
- Sounds
- Code
- Neural Networks
- …

Encoding Characters in Bits (Bytes)

❑ASCII (American Standard Code for Information Interchange)

- ➢Using 8 bits (7 useful bits) to represent a character.
- ➢0 can be represented as 0x30 in hex or 0011 0000 in binary, or 48.
- ➢A can be represented as 0x41 in hex or 0100 0001 in binary, or 65.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0x00	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
0x10	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
0x20	SP	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
0x30	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
0x40	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
0x50	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
0x60	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
0x70	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

Summary

- **All data are stored in computers as bits.**
  - Integers: Base-2 system.
  - Negative integers: Two's complement.
  - Decimals: Floating-point numbers with IEEE 754.
- **Computers are made of transistors that accept bit inputs.**
  - Composing of logic gates with transistors.
  - Composing arithmetic units with logical gates.